Slang Classifier: Team Plan and Progress

Nov 1, 2026 · @immortal

Progress so far

The project is about 34% complete: the NLP side is 60% done and the graph and delivery side is not built yet.

Area	Weight	Status	Done
Planning, synopsis, architecture	10%	Complete	100%
NLP + ML (Chiranjiv), Steps 1-10	40%	Steps 1-6 built and tested; Steps 7-10 pending (model, evaluation, JSON output, handoff)	60%
Graph engine, Neo4j, API, demo page	50%	Designed only, nothing built	0%
Overall	100%		**34%**

This is a weighted estimate (10 + 24 + 0), not a measurement. Two caveats on what counts as done:

- The dataset is a starter set of 160 rows (8 slang words, 2 meanings each, 10 sentences per meaning).
- TextBlob sentiment is noisy. It scored "The chicken is cooked" as negative, so it may be replaced later.

Three-way split

Chiranjiv keeps NLP and ML, Payal takes the graph engine, and Aditya takes the knowledge base and delivery layer.

Person	Owns
Chiranjiv: NLP + ML	Steps 7-10 (TF-IDF + Logistic Regression, evaluation, JSON output). Lead on dataset expansion. The emerging-slang module. Packaging the pip library and integration testing
Payal: graph engine	Context graph builder (NetworkX) from the JSON. k-hop, shortest path, weighted scoring and graph similarity. The meaning-score formula. Fusing the graph score with the ML score. The graph_evidence explanation
Aditya: knowledge base and delivery	Neo4j schema, seeding it from the dataset, and Cypher queries for known meaning patterns. FastAPI wrapper. Demo page with Cytoscape graph visualization

Payal's similarity step compares against the patterns Aditya stores. Until Neo4j is up, she can use a dictionary built straight from the CSV as a stand-in.

Working in parallel

1. Chiranjiv publishes the JSON contract first, as a sample output file.
2. Payal and Aditya build against that mock data from day one.
3. All three use one shared GitHub repo, with one branch per person.

Dataset expansion

The dataset needs to grow: with 10 sentences per meaning, a train/test split leaves about 2 test sentences per class, so any accuracy number would mean nothing.

- **Target size:** 15-20 slang words, 2-3 meanings each, about 40-50 sentences per meaning, roughly 1,500-3,000 rows.
- **Generated sentences:** LLM-generated sentences are quick, but they come out stereotyped and will inflate accuracy.
- **Honest test set:** keep a held-out set of about 200 sentences that the three of us collect from real social posts.

- **Splitting the work:** divide the slang words between Chiranjiv, Payal and Aditya so collection runs in parallel.

Real-time social media idea

Our guide's idea works best defined as emerging-slang discovery plus real-time disambiguation, built as an extension after the core pipeline works.

"Predict slang used on social media in real time" can mean three things:

1. **Real-time disambiguation:** run our pipeline on a live stream of posts. Easy once the core works.
2. **Emerging slang discovery:** detect new words that are not in the dictionary (a frequency spike, an unusual context) and infer their likely meaning from their neighbors in a co-occurrence graph, comparing them to known slang nodes. This is the strong part, and a real use of graph theory.
3. **Forecasting which slang will trend:** a time-series problem, which we leave out.

For the demo, replay a collected sample as a simulated stream instead of depending on live access. Platform API access and terms change often, so check current rules before building on one; open feeds such as Bluesky or Mastodon public timelines are the safest to start with.

This also means updating the synopsis scope and objectives.

Open questions

Two answers are needed before the schedule and the social media module can be planned.

- [ ] What is the submission or demo deadline?
- [ ] Did our guide name a specific platform (X, Reddit, Instagram)?

Once answered, the next deliverables are the sample JSON contract for Payal and Aditya, and the first expanded batch of the dataset.